Code Report

Software Implementation of a Governed Multi-Agent System for Autonomous E-Commerce and Accounting Operations

An evidence-based description of the source code, its architecture, its execution paths, its tests and its implementation gaps

Document type:	Software implementation (code) report
Scope:	Repository `MultiAgentAI-Research` at the time of inspection
Code inspected:	`reports/dataset_report/analysis/`, `reports/dataset_report/tests/`, `reports/dataset_selection_table/scripts/`, `reports/Agents reports/figures/`
Companion reports:	`reports/dataset_selection_table/main.tex` (project/dataset overview), `reports/dataset_report/dataset_report.tex` (Dataset Report), `reports/Agents reports/Agent_Report.tex` (Agent Report)
Diagrams:	`figures/` (generated by `figures/make_figures.py`)
Date:	October 17, 2026

Integrity note. This report documents only what exists in the repository. Inspection found ***no agent implementation****: all seven conceptual agents are recorded as* designed but not implemented*, and the only executable software is a dataset analysis, validation and reporting pipeline. Every implementation claim below is supported by a file path, a symbol name or a measured command output. Where a step could not be verified, this is stated explicitly.*

Abstract

This report documents the software that actually exists in the repository of the research project *Governed Multi-Agent AI for Autonomous E-Commerce and Accounting Operations*. Repository inspection found **2,144 lines of Python across eleven files, containing 65 functions and zero classes**, plus three LaTeX report sources. There is **no agent implementation, no orchestration layer, no message schema, no policy engine and no audit store**; the directory `src/` is empty and the root `README.md` is a zero-byte file. The implemented software is (i) a reproducible dataset-analysis and validation package under `reports/dataset_report/analysis/` with seven modules and a `python -m analysis` entry point, which reads both raw datasets read-only and writes 26 CSV files, one JSON summary and thirteen figures; (ii) an independent, standard-library dataset inspector under `reports/dataset_selection_table/scripts/inspect_datasets.py`; and (iii) two figure-generation scripts for the sibling reports. The analysis package is covered by thirteen unit tests, all of which were executed for this report and pass (`13 passed in 1.49 s`), and the full pipeline was re-executed and verified to be deterministic: all 27 generated files were reproduced with byte-identical SHA-256 digests. Inspection also produced three concrete, verified defects: a mis-detected CSV dialect in the inspector that over-counts the Olist review table by 26 rows, an empty `src/` placeholder, and the complete absence of a dependency manifest. Each of the seven conceptual agents is assessed individually and assigned the status *designed but not implemented*; where the data layer already computes a relevant primitive (for example the payment-versus-item reconciliation proxy, or IQR outlier fences) this is stated precisely and is not presented as an agent. No performance figure, no fraud-detection result and no capability is fabricated. A prioritized roadmap converts the current pipeline into a stable ingestion and validation layer on which the agents can be built.

Contents

Glossary, Abbreviations and Status Vocabulary **3**

1 Executive Summary **3**

2 Introduction and Software Scope **4**
2.1 Role of the codebase in the research project 4
2.2 Concept, proposal, implementation and plan 4
2.3 What the current code can actually execute 4

3 Development Environment and Technologies **5**

4 Repository Structure **7**

5 Software Architecture **8**
5.1 Layer by layer 8
5.2 Architectural character 9

6 Main Execution Workflow **9**
6.1 Path A — `python -m analysis` 10
6.2 Path B — `inspect_datasets.py` 11
6.3 Path C — the test suite 11

7 Detailed Code Analysis **11**
7.1 `analysis/config.py` 11
7.2 `analysis/profiling.py` 12
7.3 `analysis/olist.py` 12
7.4 `analysis/online_retail.py` 12
7.5 `analysis/compare.py` 12

7.6 analysis/figures.py . . . 13
7.7 analysis/__main__.py . . . 13
7.8 reports/dataset_report/tests/test_analysis.py . . . 13
7.9 reports/dataset_selection_table/scripts/inspect_datasets.py . . . 13
7.10 reports/Agents reports/figures/make_figures.py . . . 13

8 Implementation of the Seven Agents 13
8.1 Orchestrator / Coordination Agent . . . 14
8.2 E-Commerce Operations Agent . . . 14
8.3 Inventory Management Agent . . . 14
8.4 Accounting Agent . . . 14
8.5 Risk and Anomaly Detection Agent . . . 15
8.6 Governance and Policy Enforcement Agent . . . 15
8.7 Audit and Traceability Agent . . . 15

9 Data Pipeline and Dataset Integration 16

10 Data Contracts and Schemas 17

11 Agent Communication and Workflow Orchestration 18

12 Governance, Security and Audit Implementation 19

13 Testing and Code Quality 21

14 Execution and Reproducibility Guide 22

15 Outputs and Evaluation 23
15.1 Artifacts produced by the code . . . 23
15.2 Evaluation status and proposed plan . . . 25

16 Limitations and Technical Debt 25

17 Development Roadmap 27

18 Conclusion 28

19 References and Technical Documentation 28

A Repository Inventory and Verification Log 30

Glossary, Abbreviations and Status Vocabulary

Term	Meaning in this report
Agent	A software component with an explicit role, inputs, outputs and a governance-wrapped interface.
Audit event	A record of an action and its decision context. **No audit event is produced by the current code.**
BRL	Brazilian Real, the currency implied by the Olist marketplace context.
Entry point	A module that can be started directly: `python -m analysis` or `python .../inspect_datasets.py`.
Gate	A validation check that raises on failure and therefore aborts the run (fail-closed).
GBP	Pound sterling, the currency of Online Retail II.
Inspector	The standard-library script `inspect_datasets.py` in `reports/dataset_selection_table/scripts/`.
Provenance manifest	The SHA-256-per-file listing written to `dataset_inventory.csv`. It is *not* an audit trail.
Implemented	Present as executable code, evidenced by a file path and a symbol name.
Partially implemented	Implemented in a narrow form, with a named limitation; never described as complete.
Designed but not implemented	Specified in the sibling reports or in qualitative CSV columns, but with no executable code.
Not found in repository	No design and no code; used only where the design also does not exist.

1 Executive Summary

Software objective. The codebase is intended to become the executable substrate of a governed multi-agent system [28, 21, 20] that coordinates e-commerce operations, inventory analysis, accounting tasks, anomaly detection, governance enforcement and audit traceability. At present the codebase implements the *front* of that system: a reproducible pipeline that ingests the two research datasets, validates their structure, profiles them, derives a small set of defensible aggregate facts, and exports machine-readable and human-readable evidence for the sibling reports.

What kind of system the repository contains. Inspection is unambiguous: the repository contains a **dataset analysis and validation pipeline together with a documentation and reporting pipeline**. It does *not* contain a working multi-agent system, nor even a partial prototype of one. There are no agent classes — in fact there are no class definitions anywhere in the repository — no orchestrator, no message bus, no policy engine and no audit store. The directory `src/` exists but is empty, and the root `README.md` is empty. Consequently all seven conceptual agents are classified as *designed but not implemented* (Section 8).

Main implemented modules. The substantive implementation is the Python package `reports/dataset_report/analysis/`, comprising `config`, `profiling`, `olist`, `online_retail`, `compare`, `figures` and the `__main__` entry point. A second, independent program, `reports/dataset_selection_table/scripts/inspect_datasets.py`, performs a standard-library inventory and quality scan of the same raw files, and `reports/Agents reports/figures/make_figures.py` renders the schematic diagrams of the Agent Report. Table 3 quantifies all Python in the repository.

Technologies. Python 3.10.11 [19] with pandas 2.3.3 [13, 23], NumPy 1.26.4 [8, 16], matplotlib 3.10.7 [10, 12], seaborn 0.13.2 [25, 26], openpyxl 3.1.5 [7] and pytest 9.1.0 [18], plus the Python standard library; MiKTeX-pdfTeX 4.18 with BibTeX 4.1 [14] for the LaTeX reports (Section 3). No LLM framework, no agent-orchestration library, no machine-learning library, no API client and no database driver are used.

Implementation maturity. The data layer is mature enough to be trustworthy: it is deterministic, it fails closed on two structural violations, it is unit-tested at the helper level, and its thirteen tests pass. The agent layer has zero maturity: nothing exists. Governance is enforced only to the extent that the data layer validates columns and timestamps; no authorization, policy decision, approval or audit event exists.

Principal limitations. (i) The seven agents, orchestration, governance enforcement and audit are absent. (ii) There is no dependency manifest, so the environment is not pinned. (iii) The inspector contains a verified CSV-dialect defect that over-counts the Olist review table by 26 rows (99,250 instead of 99,224), which propagates into the dataset-selection artifacts and into `main.tex`. (iv) The code has no logging, no output validation, no CLI for the analysis package, and no integration or end-to-end tests. These are documented with evidence in Section 16.

2 Introduction and Software Scope

2.1 Role of the codebase in the research project

The research project studies how governed, specialised software agents can cooperate on e-commerce and accounting workflows while remaining traceable, a design-science undertaking in the sense of Hevner et al. [9]. Four artefacts support that study: the project/dataset overview (delivered as `reports/dataset_selection_table/main.tex`; no file named literally `Main_Report` exists in the repository), the Dataset Report (`reports/dataset_report/dataset_report.tex`), the Agent Report (`reports/Agents reports/Agent_Report.tex`), and this Code Report. This document is the only one that concentrates on *software*: how the code is organised, what it executes, how data flows through it, how it is tested, and what remains to be built. Dataset statistics are therefore not repeated here; the reader is referred to the Dataset Report [17, 4]. Agent responsibilities, the governance lifecycle and the proposed data contracts are not repeated either; the reader is referred to the Agent Report.

2.2 Concept, proposal, implementation and plan

The task distinguishes four things, and this report keeps them separate at all times.

- **Research concept.** A governed multi-agent system for autonomous e-commerce and accounting operations, with seven agent roles (Section 8).
- **Proposed architecture.** The layered design, the governance lifecycle, the audit event schema and the evaluation protocol specified in the sibling reports. These are *design documents*, not code.
- **Implemented software.** The dataset analysis and validation package, the stdlib inspector and the figure generators, all listed in Table 2 and analysed in Section 7.
- **Planned features.** The roadmap of Section 17.

2.3 What the current code can actually execute

Running the code today performs exactly four operations, all verified during preparation of this report:

1. `python -m analysis` (from `reports/dataset_report`) reads the nine Olist CSV files and both Online Retail II worksheets read-only, computes profiling, key, referential-integrity, join, distribution, temporal and monetary metrics, derives a payment-versus-item reconciliation proxy and several retail business signals, writes 26 CSV files and one JSON summary into `analysis_results/`, and renders thirteen PNG figures into `figures/`.
2. `python reports/dataset_selection_table/scripts/inspect_datasets.py` (from the repository root) performs an independent, standard-library inventory and quality scan and writes five CSV artifacts plus an `inspection_summary.json`.
3. `python figures/make_figures.py` (in `reports/Agents reports/figures` or in `reports/code_report/figures`) renders schematic diagrams for the respective report.
4. `python -m pytest tests -q` (from `reports/dataset_report`) executes thirteen unit tests.

Nothing in the repository accepts a business request, plans a task, selects an agent, evaluates a policy, requests approval or records an audit event. Those operations are not implemented in any form.

3 Development Environment and Technologies

The environment was characterised by direct runtime inspection rather than by reading a manifest, because **no dependency manifest exists**: there is no `requirements.txt`, `environment.yml`, `pyproject.toml`, `setup.py`, `Pipfile` or lock file anywhere in the repository. Versions below were obtained with `python -version`, `pip list`, `pytest -version` and `pdflatex -version` in the working environment, and cross-checked against imports actually present in the source and against the compiled byte-code file names (for example `test_analysis.cpython-310-pytest-9.1.0.pyc`). Where a version could not be established this is stated.

Technology	Version	Purpose in this repository	Evidence in repository
Python	3.10.11	Language runtime for all code	`python -version`; byte-code names `*.cpython-310*.pyc` in `analysis/__pycache__`- and `tests/__pycache__`
pandas	2.3.3	CSV/Excel loading, profiling, joins, aggregation	`import pandas as pd` in `analysis/config.py`, `profiling.py`, `olist.py`, `online_retail.py`, `__main_-_`; `pip list`
NumPy	1.26.4	numeric dtypes, quantiles, IQR fences, NaN handling	`import numpy as np` in `profiling.py`, `olist.py`, `online_retail.py`, `figures.py`
matplotlib	3.10.7	figure rendering (Agg backend)	`matplotlib.use("Agg")` in `analysis/figures.py` and `figures/make_figures.py`
seaborn	0.13.2	plot theme and statistical plots	`import seaborn as sns` in `analysis/figures.py`
openpyxl	3.1.5	Excel engine for `pandas.read_excel`	`engine="openpyxl"` in `online_retail.load_-sheets()`
pytest	9.1.0	unit-test runner	`tests/test_analysis.py`; `pytest -version`; `*.pytest-9.1.0.pyc`
Python standard library	3.10.11	inspector, hashing, I/O helpers	`csv`, `zipfile`, `xml.etree.ElementTree`, `hashlib`, `argparse`, `json`, `pathlib` in `inspect_-datasets.py` and `config.py`
LaTeX (MiKTeX-pdfTeX)	4.18 (MiKTeX 24.1)	compilation of report sources	`pdflatex -version`; `*.tex` and `*.pdf` in `reports/`
BibTeX	4.1 (MiKTeX 24.1)	bibliography processing	`bibtex -version`; `references.bib` and `*.bbl` in `reports/`

Table 1: Technologies verified to be used by the repository's code, with the file evidence. No library is listed merely because it is common in AI projects.

Technologies deliberately absent. The following were checked and are **not used**: LLM and agent frameworks (no `langchain`, `langgraph`, `autogen`, `crewai`, `openai` or `anthropic` import exists, and none is installed); machine-learning or anomaly-detection libraries (no `scikit-learn`); API/client libraries; database drivers; web frameworks; workflow engines; task queues; and configuration/secrets managers. Packages such as `pydantic` 2.13.4 and `scipy` 1.15.3 are present in the working environment but are imported by no repository file, so they are not project dependencies either. This absence is the single most important environmental fact for this report: there is no infrastructure in which an agent could run.

Operating-system assumptions. All code is platform-independent Python. Two practical Windows observations apply: the sibling READMEs document commands for Windows PowerShell, and one path containing a space (`reports/Agents reports/`) must be quoted on any command line. The analysis package derives all paths from `__file__`, so it does not depend on the current working directory beyond the requirement to run it as a module from `reports/dataset_-report` (Section 14).

4 Repository Structure

Table 2 describes every relevant file and directory. Generated caches, byte-code directories and the raw datasets themselves are excluded from the table and, in the case of the raw files, are described under "External data".

File or directory	Responsibility	Main dependencies	Status
`README.md`	None; file is empty (0 bytes)	—	Empty
`src/`	Intended source root; **empty directory**	—	Placeholder
`datasets/olist/`	Nine Olist CSV tables (read-only input)	—	External data
`datasets/online_retail_ii/online_retail_II.xlsx`	Two-worksheet Excel workbook (read-only input)	—	External data
`reports/dataset_report/analysis/config.py`	Paths, file hashing, deterministic writers	`hashlib`, `json`, `pathlib`, pandas	Implemented
`.../analysis/profiling.py`	Seven dataset-agnostic profiling helpers	pandas, NumPy	Implemented
`.../analysis/olist.py`	Olist loading, schema validation, key/RI/join/monetary analysis	pandas, NumPy, `config`, `profiling`	Implemented
`.../analysis/online_retail.py`	Excel loading, cleaning, quality and business-signal metrics	pandas, NumPy, `config`, `profiling`	Implemented
`.../analysis/compare.py`	Inventory, comparative table, agent-suitability matrix	pandas, `config`	Implemented (mixed measured + curated text)
`.../analysis/figures.py`	Thirteen analysis figures	matplotlib, seaborn, `config`	Implemented
`.../analysis/__main__.py`	Entry point `main()`; orchestration of the pipeline	all of the above	Implemented
`.../analysis/__init__.py`	Package docstring and `__all__`	—	Implemented
`.../tests/test_analysis.py`	Thirteen unit tests of helper definitions	pytest, NumPy, pandas	Implemented
`.../analysis_results/`	26 CSV artifacts + `summary.json`	generated by `analysis`	Generated artifact
`.../figures/`	Thirteen PNG analysis figures	generated by `analysis`	Generated artifact
`reports/dataset_report/dataset_report.tex/.pdf`	Dataset Report (companion)	LaTeX	Documentation
`reports/dataset_selection_table/scripts/inspect_datasets.py`	Independent stdlib inventory and quality scanner	stdlib only	Implemented
`reports/dataset_selection_table/main.tex/.pdf`	Project and dataset overview (companion; no `Main_Report` file exists)	LaTeX	Documentation
`reports/dataset_selection_table/*.csv, inspection_summary.json`	Inspector artifacts	generated	Generated artifact
`reports/Agents reports/Agent_Report.tex/.pdf`	Agent design and implementation-status report	LaTeX	Documentation
`reports/Agents reports/figures/make_figures.py`	Seven schematic PNGs for the Agent Report	matplotlib	Implemented

Table 2: Relevant repository contents and their status. Byte-code caches, `.pytest_cache` directories and the large raw data files are omitted. **Bold** entries indicate findings that matter for the roadmap.

Not present. There is no `requirements.txt`, `pyproject.toml`, `setup.py`, skill file, container definition, Makefile, `tox.ini`, `pytest.ini`, `conftest.py`, CI workflow, `.gitignore`, database schema, API module, agent module, prompt file, model configuration, or secrets file. A targeted search for credential-like identifiers (API keys, tokens, passwords) found no configuration or secret material in the repository; the only pattern matches were ordinary Portuguese words inside Olist customer-review text files. No credentials therefore need to be, or can be, reported.

5 Software Architecture

The implemented architecture is a single-process, offline, batch pipeline. Figure 1 shows the actual module relationships and simultaneously marks the proposed components that have no implementation.

Figure 1 - Repository and module architecture (solid = implemented, dashed = proposed)

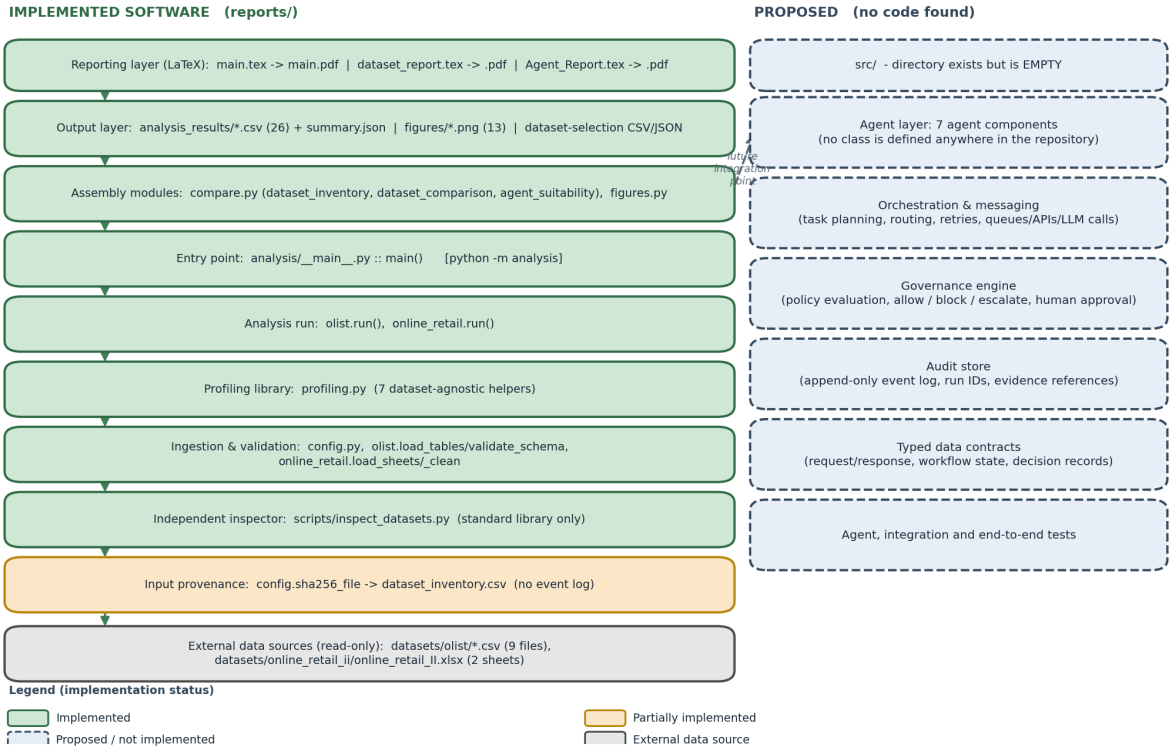


Figure 1: Repository and module architecture as observed. Green solid boxes are implemented code; amber marks the partially implemented provenance mechanism; dashed boxes are proposed and have no implementation in the repository; grey marks external read-only data.

5.1 Layer by layer

- **Application entry point.** `analysis/__main__.py`, function `main()`, executed as `python -m analysis` from `reports/dataset_report`. It calls every other module in a fixed order and is the only orchestrator that exists. The inspector has its own `main()` reached through `argparse`.
- **Configuration layer.** `analysis/config.py` derives `REPO_ROOT`, `OLIST_DIR`, `RETAIL_XLSX`, `RESULTS_DIR` and `FIGURES_DIR` from `__file__`, declares the input inventory (`OLIST_FILES`, `RETAIL_SHEETS`, `OLIST_TABLE_ORDER`) and provides `ensure_dirs()`, `sha256_file()`, `write_csv()`, `write_json()` and `round_pct()`.
- **Data access layer.** `olist.load_tables()` (`pd.read_csv`) and `online_retail.load_sheets()` (`pd.read_excel`) are the only readers. Both are strictly read-only. The inspector

implements its own readers, including a pure-Python OOXML reader `xlsx_rows()`.

- **Validation layer.** `olist.validate_schema()` compares column *sets* against `EXPECTED_COLUMNS`; `olist.load_tables()` aborts if timestamp parsing would create new `NaT`; `online_retail._clean()` aborts unless the sheet columns equal `EXPECTED_COLS` exactly. These are the only enforced gates in the repository.
- **Processing and analysis layer.** `profiling.py` supplies seven reusable helpers; `olist.run()` and `online_retail.run()` compose them with dataset-specific key, referential-integrity, join, distribution, temporal, monetary and business-signal logic.
- **Agent layer. Not implemented.** No class exists anywhere in the repository; `src/` is empty.
- **Orchestration layer. Not implemented** beyond the linear sequence inside `main()`. There is no task decomposition, routing, dependency resolution, retry or consolidation.
- **Governance layer. Not implemented.** Column and type validation exist but are data-quality controls, not policy enforcement: there is no authorization step, no policy rule set, no allow/block/escalate decision and no approval gate.
- **Audit and logging layer. Not implemented**, except for one partial primitive: `config.sha256_file()` records a SHA-256 digest per source file into `dataset_inventory.csv`, which is an input-integrity manifest. There is no event log, no run identifier, no actor, no decision record and no persistence layer. The Python `logging` module is never imported; progress is written with `print()`.
- **API or user interface layer. Not implemented.** The only interface is the command line plus the generated PDF reports.
- **Output and reporting layer.** Implemented: `config.write_csv()`, `config.write_json()`, `figures.py` and `compare.py` produce the artifacts consumed by the LaTeX sources.

5.2 Architectural character

The design is a strict batch DAG executed in a fixed order with no branching in the happy path: load → validate → profile → analyse → serialise → plot. Data crosses module boundaries as plain `dicts` of `pandas DataFrames` (the `frames/facts` convention). There is no asynchronous behaviour, no concurrency, no persistence beyond file writes and no external service interaction. The architecture is appropriate for its current purpose (reproducible dataset analysis) and is, by construction, the opposite of what a multi-agent system requires.

6 Main Execution Workflow

There are three execution paths. Figure 2 shows all three, including the two fail-closed gates.

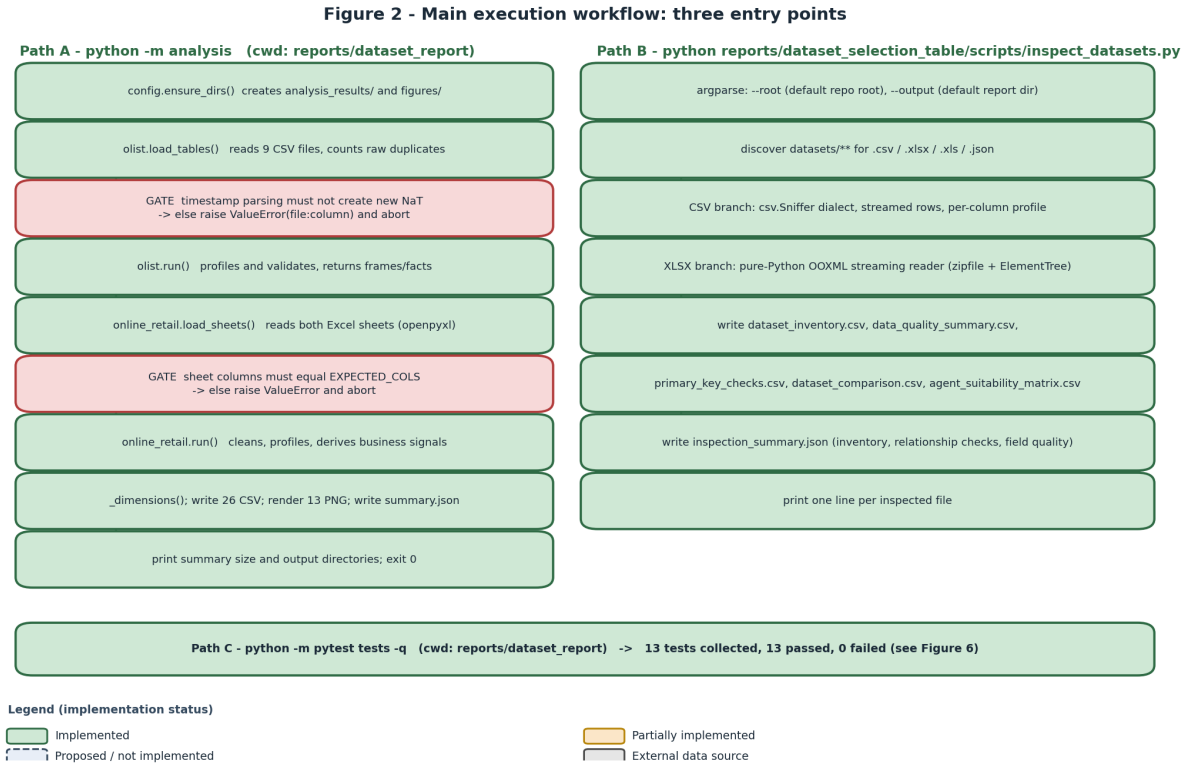


Figure 2: The three real execution paths. Path A is the analysis package, Path B the standard-library inspector, Path C the test suite. Amber boxes are validation gates that raise and abort the run.

6.1 Path A — `python -m analysis`

1. **Start.** The interpreter imports the package `analysis` and executes `analysis/__main__.py`.
2. **Configuration.** `config.ensure_dirs()` creates `analysis_results/` and `figures/` if absent. All other configuration is compile-time constants in `config.py`; nothing is read from the environment or a file.
3. **Input selection.** `olist.load_tables()` iterates the fixed list `config.OLIST_FILES`; `online_retail.load_sheets()` reads all sheets of `config.RETAIL_XLSX`. There is no argument to select or filter inputs.
4. **Validation and processing.** For each Olist table the loader counts exact-duplicate rows on the raw values, then parses the columns named in `DATE_COLUMNS` and *raises* if parsing would turn a previously present value into `NaT`. `olist.run()` then validates the schema, profiles every table, and runs key checks, referential-integrity checks, join-cardinality and join-inflation measurements, distributions, temporal-integrity checks, monetary summaries and the payment-versus-item reconciliation proxy. For the workbook, `online_retail._clean()` enforces the exact column list and then normalises codes, strings, dates and numerics, preserving genuine missing values as `pd.NA`.
5. **Invocation of components.** All invocations are direct Python calls from `main()`; see Section 11.
6. **Output validation. None.** No assertion, checksum, schema check or row-count check is applied to the generated artifacts. This is a real gap and is listed in Section 16.
7. **Error handling.** Only the two gates above raise; every other operation relies on pandas' `errors="coerce"` semantics or on coercion in the loaders. There is no `try`/`except` block anywhere in the analysis package except the single `TypeError` guard in `profiling.py` (Section 7), so any other unexpected exception terminates the run with a traceback and leaves the previously written artifacts in place.

8. **Result storage.** CSV artifacts via `config.write_csv()` (with `float_format="%.6g"` for determinism), the JSON summary via `config.write_json()`, and PNG figures via `figures._save()` at 150 dpi. `main()` finishes by printing a truncated copy of the summary and the two output directories.

6.2 Path B — `inspect_datasets.py`

Execution starts in `main()`, which parses `-root` and `-output` with `argparse`, then discovers every `.csv`/`.xlsx`/`.xls`/`.json` file under `<root>/datasets`. CSV files are read with `csv.Sniffer` dialect detection and streamed row by row, accumulating per-column null counts, distinct sets, numeric ranges, timestamp ranges, SHA-256 row hashes for duplicate detection and composite-key checks. The Excel workbook is read without third-party libraries through `xlsx_rows()`, which opens the file as a ZIP archive and streams `xl/worksheets/sheet*.xml` with `ElementTree`. The run writes five CSV artifacts and `inspection_summary.json`, then prints one line per file. Error handling is defensive rather than fail-closed: the type and date parsers use bare `except` clauses that silently skip values (Section 16).

6.3 Path C — the test suite

`python -m pytest tests -q` from `reports/dataset_report` collects `tests/test_analysis.py`, which inserts the package's parent directory on `sys.path` and exercises helper definitions with small in-memory frames. It never reads the raw datasets.

7 Detailed Code Analysis

This section documents each significant module. Table 3 gives the measured size of every Python file. "Functions" counts `def` statements, including private helpers and test functions.

Python file	Lines	Functions	Classes
reports/Agents	380	10	0
reports/figures/make_figures.py			
reports/dataset_report/analysis/olist.py	381	5	0
reports/dataset_report/analysis/online_-	230	8	0
retail.py			
reports/dataset_report/analysis/profiling.py	217	8	0
reports/dataset_report/analysis/figures.py	202	4	0
reports/dataset_report/analysis/compare.py	142	3	0
reports/dataset_report/analysis/__main__.py	123	2	0
reports/dataset_report/analysis/config.py	88	5	0
reports/dataset_report/analysis/__init__.py	6	0	0
reports/dataset_report/tests/test_analysis.py	177	14	0
reports/dataset_selection_-	198	6	0
table/scripts/inspect_datasets.py			
Total	**2,144**	**65**	**0**

Table 3: Measured size of the Python code. The repository contains **no class definitions at all**, which is decisive evidence that no agent is implemented.

7.1 `analysis/config.py`

Purpose: single source of truth for paths, the input inventory and deterministic writers. *Key symbols:* module constants `PACKAGE_DIR`, `REPORT_DIR`, `REPO_ROOT`, `OLIST_DIR`, `RETAIL_XLSX`, `RESULTS_DIR`, `FIGURES_DIR`, `OLIST_FILES`, `OLIST_TABLE_ORDER`, `RETAIL_SHEETS`; functions `ensure_dirs`, `sha256_file`, `write_csv`, `write_json`, `round_pct`. *Inputs/outputs:* filesystem paths

in, `Path` objects and hashes out. *Validation:* none. *Notable design decision:* `write_csv` fixes `float_format="%.6g"`, which is the mechanism that makes the pipeline outputs reproducible; this was confirmed empirically (Section 13).

7.2 analysis/profiling.py

Purpose: dataset-agnostic descriptive statistics. *Key symbols:* `column_profile`, `exact_duplicates`, `numeric_stats`, `categorical_stats`, `value_counts_table`, `temporal_profile`, `key_check`, plus the private `_is_datetime_like`. *Processing logic:* `numeric_stats` computes count, mean, standard deviation, quartiles, extremes, zero and negative counts, and Tukey IQR fences with an outlier count (this is the strongest anomaly-adjacent primitive in the repository, but it is a *descriptive statistic*, not a detector). `value_counts_table` preserves missingness as the explicit category `<missing>` instead of silently stringifying it. `key_check` reports missing key components, distinct key values and duplicate-excess rows. *Validation:* only a `TypeError` guard around `nunique`. *Module docstring commitment:* metrics are descriptive and "no values are imputed or invented".

7.3 analysis/olist.py

Purpose: the Olist-specific pipeline. *Key symbols:* constants `EXPECTED_COLUMNS` (nine tables), `DATE_COLUMNS`; functions `load_tables`, `validate_schema`, `referential_integrity`, `_join_expansion`, `run`. *Error handling:* `load_tables` contains the only fail-closed timestamp gate:

```
newly_missing = int((before.notna() & df[col].isna()).sum())
if newly_missing:
    raise ValueError(f"{fname}:{col}: {newly_missing} values failed "
                     "timestamp parsing")
```

Notable logic: `referential_integrity` is deliberately a pure module-level function so it can be unit-tested without the datasets; `run` derives `line_total = price + freight` and computes the order-level reconciliation proxy as the absolute difference between summed `payment_value` and summed line totals, reporting the share of orders matching within one cent. *Status:* fully implemented for the metrics it claims.

7.4 analysis/online_retail.py

Purpose: the workbook pipeline, including the only cleaning layer in the repository. *Key symbols:* `EXPECTED_COLS`, `load_sheets`, `_clean_code`, `_strip_nullable`, `_clean`, `_invoice_kind`, `_sheet_quality`, `run`. *Validation:* `_clean` raises if the sheet columns differ from the expected eight. *Notable logic:* `_clean_code` exists because Excel stores some stock and invoice codes as floats (for example `85123.0`) and others as strings (`85123A`, `C489449`); it strips the spurious `.0` while mapping blanks to `pd.NA`. `_invoice_kind` defines a cancellation as any invoice number beginning with `C`. `_sheet_quality` computes the business and data-quality signals used by the report (cancellation rows and invoices, negative and zero quantities, zero and negative prices, missing customer identifiers, gross positive value, cancellation value, net revenue). The module docstring is explicit that these are signals and "not treated as fraud". *Status:* implemented; note that only column-name validation is applied here, in contrast with the richer Olist validation.

7.5 analysis/compare.py

Purpose: assemble cross-dataset artifacts. *Key symbols:* `dataset_inventory`, `dataset_comparison`, `agent_suitability`. *Important caveat:* `dataset_comparison` and `agent_suitability` mix measured values with manually curated qualitative text; the module docstring states that "the qualitative columns are manually curated". They should therefore be read as curated analysis,

not as computed output. `dataset_inventory` is the only place where `sha256_file` is used, producing the input-integrity manifest, a small step towards the reproducible, well-described data stewardship advocated by the FAIR principles [27].

7.6 `analysis/figures.py`

Purpose: convert frames into thirteen PNGs. *Key symbols:* `_save`, `olist_figures`, `retail_figures`, `comparison_figure`. *Processing logic:* selects the Agg backend at import, sets a seaborn theme, and plots monthly order volume, status, payment types, review scores, category revenue, missingness, retail monthly lines and net revenue, top countries, log line-value distribution, quality signals, cancellation rate and a completeness comparison. *Design note:* importing this module has the global side effect of selecting a matplotlib backend and applying a seaborn theme.

7.7 `analysis/__main__.py`

Purpose: the pipeline orchestrator. *Key symbols:* `_dimensions`, `main`. *Behaviour:* calls `config.ensure_dirs`, the two loaders, the two `run` functions, `_dimensions`, twenty-six `write_csv` calls, `write_json`, the three figure functions, and finally prints a truncated summary. It contains no argument parsing, no logging and no try/except.

7.8 `reports/dataset_report/tests/test_analysis.py`

Purpose: unit-test the definitions of the derived metrics. *Symbols:* fourteen test/builder functions; the helper `_retail_frame()` builds a five-row synthetic frame. *Inputs:* in-memory frames only. *Status:* implemented and passing (Section 13).

7.9 `reports/dataset_selection_table/scripts/inspect_datasets.py`

Purpose: an independent, dependency-free inventory and quality scan. *Key symbols:* `xlsx_rows`, `typ`, `pct`, `type_hint`, `main`, plus the local closure `vals`. *Design decision:* it deliberately avoids pandas and openpyxl so that the inspection can run in a bare Python 3.9+ environment. *Verified defects:* (i) `csv.Sniffer` is used without forcing `doublequote`, and the sniffed dialect reports `doublequote=False`, which mis-parses rows containing escaped quotes and inflates the Olist review table to 99,250 rows instead of 99,224 (Section 16); (ii) line 99 computes `tcount = Counter(...)` over an empty generator and never uses it, so it is dead code; (iii) numerous bare `except` clauses suppress parsing errors; (iv) `ROOT` is derived as `parents[3]`, which is correct only while the script stays at its present depth.

7.10 `reports/Agents reports/figures/make_figures.py`

Purpose: render seven schematic diagrams for the Agent Report. *Key symbols:* `_new_axes`, `box`, `arrow`, `legend`, `architecture`, `status_matrix`, `governance_lifecycle`, `sequence`, `scenarios`, `main`. *Status:* implemented; it is a drawing utility and computes no metrics. Its docstring correctly states that the diagrams are "schematic representations of the PROPOSED architecture".

8 Implementation of the Seven Agents

This section treats each conceptual agent separately and assigns an implementation status using the vocabulary defined in the glossary. The headline finding is that **no agent is implemented**. The impact is concrete: the repository contains zero class definitions, zero message definitions, zero policy rules and zero audit records. Table 4 summarises; the subsections that follow give the evidence and, importantly, identify the data-layer primitives that already exist and could be reused, so that the roadmap does not recommend rewriting working code.

Conceptual agent	Status	Existing data-layer precursor (if any)
8.1 Orchestrator / Coordination	Designed but not implemented	Linear sequence in `main()` only
8.2 E-Commerce Operations	Designed but not implemented	`olist.run()` aggregates orders, status, payments and reviews
8.3 Inventory Management	Designed but not implemented	Category revenue and proton/line quantities; **no stock data exists**
8.4 Accounting	Designed but not implemented	Payment-versus-item reconciliation proxy in `olist.run()`
8.5 Risk and Anomaly Detection	Designed but not implemented	IQR outlier fences; negative-quantity and zero-price counts
8.6 Governance and Policy Enforcement	Designed but not implemented	Column and timestamp validation gates only
8.7 Audit and Traceability	Designed but not implemented	SHA-256 input manifest only; no event log

Table 4: Implementation status of the seven conceptual agents, determined from repository evidence. No agent has executable code.

8.1 Orchestrator / Coordination Agent

Status: designed but not implemented. The only coordination in the repository is the fixed call sequence inside `analysis/__main__.py::main()` (lines 50–119). There is no task object, no decomposition, no assignment, no dependency graph, no state machine, no failure recovery and no result consolidation. Nothing receives or routes a task. The sequential analyse-then-export order is the closest analogue that exists, and Figure 3 shows it in the green band.

8.2 E-Commerce Operations Agent

Status: designed but not implemented. `olist.run()` computes order-status distributions, monthly order counts, payment-type distributions, review-score distributions and a category-revenue ranking; `online_retail.run()` computes monthly transaction lines, monthly net revenue, top countries and cancellation rates. These are aggregate metrics of the kind such an agent would consume, but no component accepts an order, customer or product query and returns a decision or a recommendation.

8.3 Inventory Management Agent

Status: designed but not implemented. This is the most data-constrained agent. Neither dataset contains stock-on-hand quantities, inventory movements, receipts, reservations or replenishment events; `compare.agent_suitability()` states this explicitly for both sources. The code can rank products by `line_total` or category but cannot compute an inventory position. Any inventory claim beyond demand indicators would be unsupported by the available data, and the implementation correctly makes none.

8.4 Accounting Agent

Status: designed but not implemented. There is *no* ledger, no chart of accounts, no journal entry, no tax logic and no double-entry structure in the repository or in the data, so none of the internal-control objectives that frameworks such as COSO [5] and reporting regimes such as the Sarbanes–Oxley Act [24] presuppose can be met. What does exist is a genuinely useful metric in `olist.run()`: per order, the summed `payment_value` is compared with the summed `line_total`, and the share of orders agreeing within one cent is reported. This is a reconciliation *proxy*, and it is labelled as such in the code and in the Dataset Report; it is not a financial control.

8.5 Risk and Anomaly Detection Agent

Status: designed but not implemented. `profiling.numeric_stats()` computes Tukey IQR fences and counts points outside them, and `online_retail._sheet_quality()` counts negative quantities, zero and negative prices, cancellation lines and non-numeric invoice identifiers. These are transparent, rule-based, statistical descriptors of the kind catalogued in the anomaly-detection literature [3, 15]. There is no model, no threshold configuration, no feature pipeline and, critically, **no ground-truth fraud label in either dataset** — `agent_suitability()` says so directly. Consequently no precision, recall or fraud-detection performance can be claimed, and none is claimed here. The correct description of these outputs is "candidate anomalies for human review".

8.6 Governance and Policy Enforcement Agent

Status: designed but not implemented. Governance is described only in the sibling reports and in the qualitative `agent_suitability` columns. The code performs *data* validation (schema set-equality, exact column lists, timestamp non-regression) and one partial control (the SHA-256 input manifest), but it has no notion of authorization, permission, policy rule, threshold owner, approval, block or escalation. There is no identity, no role and no audit of a decision. The proposed governance lifecycle in the Agent Report therefore remains a design artefact.

8.7 Audit and Traceability Agent

Status: designed but not implemented. The only traceability primitive is `config.sha256_file()`, whose output is written by `compare.dataset_inventory()` into `dataset_inventory.csv`. That manifest allows a reader to verify that the same input bytes were used, which supports reproducibility, but it is not an audit trail: there is no run identifier, no timestamp, no actor, no action, no decision and no append-only event store. The system must not be described as immutable, tamper-proof or cryptographically verifiable; the code provides no such property.

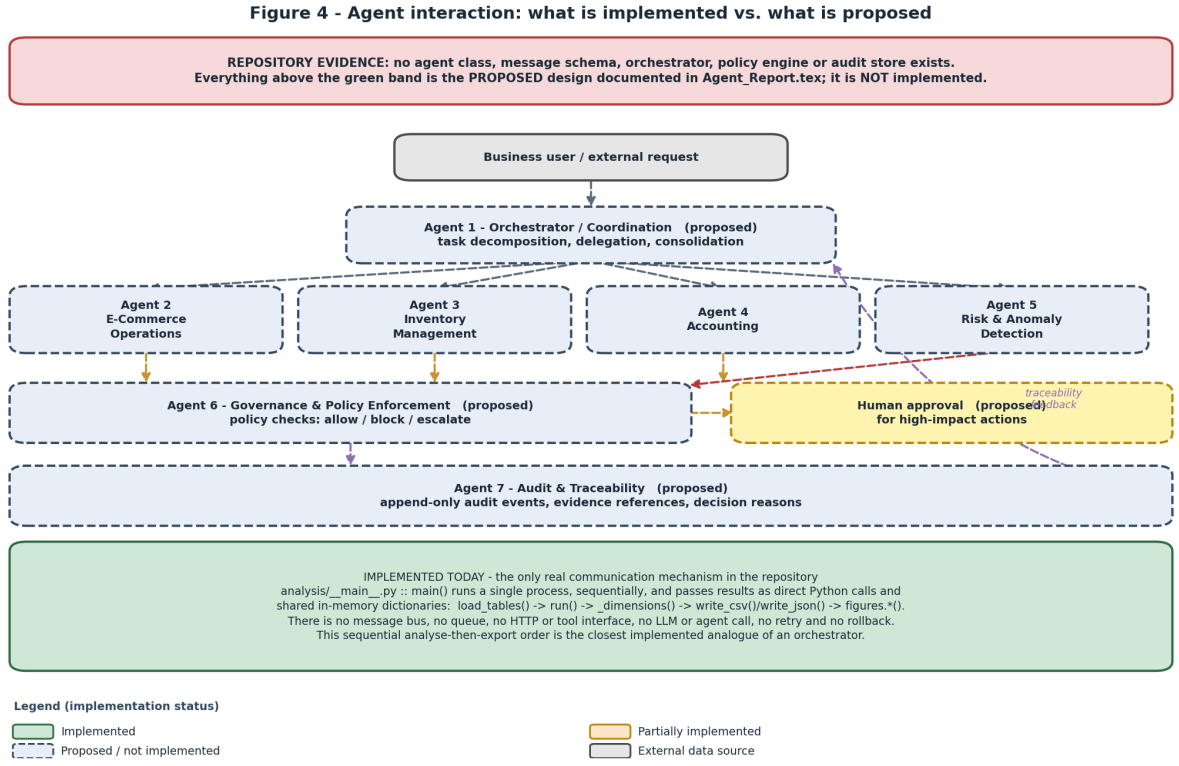


Figure 3: Agent interaction. Everything above the green band is the proposed design; the green band is the only implemented communication mechanism, namely direct Python calls in a single sequential process.

9 Data Pipeline and Dataset Integration

Figure 4 presents the pipeline as implemented. The reader is referred to the Dataset Report for dataset characteristics; only the code's operations are described here.

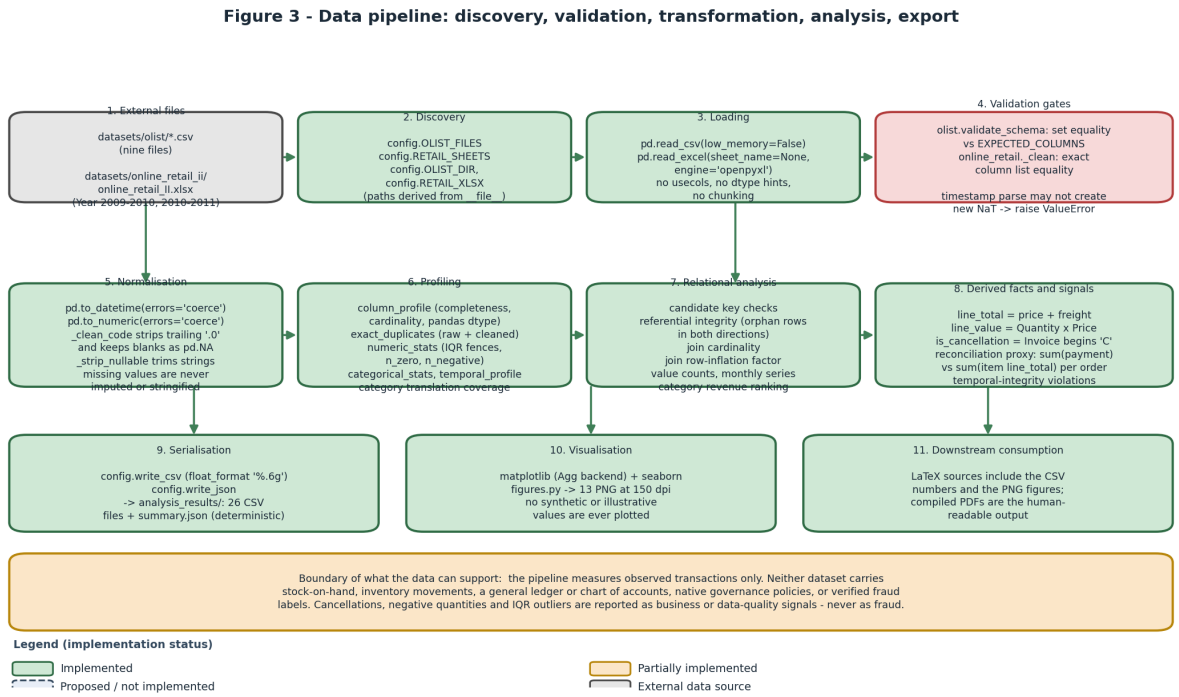


Figure 4: The implemented data pipeline. Red marks the validation gates that abort the run on failure; amber marks the explicit statement of what the data cannot support.

Input discovery and loading. `config.OLIST_FILES` names the nine CSVs and `config.RETAIL_SHEETS` names the two worksheets. Olist files are read with `pd.read_csv(..., low_memory=False)`; the workbook is read with `pd.read_excel(..., sheet_name=None, engine="openpyxl")`, which loads both sheets into memory. No `usecols`, `dtype`, chunk size or `parse_dates` argument is supplied.

Formats and paths. CSV with a UTF-8 BOM in the case of the inspector (opened with `encoding="utf-8-sig"`) and XLSX for the workbook. Paths are derived from `__file__`, so the package works from any working directory as long as it is invoked as a module from `reports/dataset_report`.

Column selection and type conversion. No columns are dropped. In Olist, only the columns listed in `DATE_COLUMNS` are converted (to `datetime64`), protected by the non-regression gate. In the workbook, `_clean` converts `InvoiceDate`, `Quantity`, `Price` and `Customer ID` with `errors="coerce"` and normalises the code and text columns.

Missing-value handling. This is a deliberate design point consistent with established data-quality practice [2]: missing values are preserved, never imputed and never stringified. `_clean_code` maps blanks to `pd.NA`; `_strip_nullable` restores real missingness after a string operation; `value_counts_table` reports an explicit `<missing>` category; `column_profile` reports `n_missing` and `missing_pct`. Four dedicated tests assert these behaviours.

Duplicate detection. `exact_duplicates()` counts exact duplicate rows; `olist.load_tables()` separately records raw duplicates before parsing so that "original-data duplicates" and "parsed duplicates" remain distinguishable; the workbook reports duplicates both raw and after cleaning, plus repeated line keys.

Identifier validation. `key_check()` is applied to eleven candidate keys (single and composite, including `order_id+order_item_id` and `order_id+payment_sequential`). `referential_integrity()` checks six foreign-key relationships in both directions, reporting orphan rows and orphan percentages.

Transformations and joins. Two derived fields are created: `line_total = price + freight` for Olist order items and `line_value = Quantity * Price` for the workbook. Joins are measured rather than materialised for analysis: `_join_expansion()` performs each candidate inner join only to record resulting row counts and an inflation factor, and `join_cardinality` characterises each relationship.

Feature engineering. Only the two derived value fields plus the cancellation flag and period columns (`month`) are engineered. No lag, rolling-window, frequency or basket features are produced.

Output generation. Twenty-six CSV files and one JSON summary, plus thirteen PNGs. The summary embeds the `facts` dictionaries for both datasets and the list of figure paths.

Operations performed by code versus recommendations. Everything listed above is executed by code. Two categories are *not* executed and appear only as recommendations in the sibling reports: (i) canonical-event mapping and source adapters, and (ii) any inventory-position or ledger reconstruction. The code correctly refuses to assume information the data does not contain — no stock levels, no ledger entries, no fraud labels.

A verified cross-tool inconsistency. The two independent readers disagree about the size of one table. The pandas-based loader reports 99,224 rows for `olist_order_reviews_dataset.csv`; the standard-library inspector reports 99,250. The cause was isolated: `csv.Sniffer().sniff(...)` infers a dialect with `doublequote=False`, so rows containing escaped quotes are split and embedded newlines produce 26 phantom records. The pandas count is the correct one (a plain `csv.DictReader` with the default dialect also yields 99,224). The incorrect figure appears in the inspector artifacts and in `main.tex`, and the resulting Olist total (1,550,948) differs from the Dataset Report total (1,550,922). This is reported as a defect, with the fix listed in Section 16.

10 Data Contracts and Schemas

What exists: implicit contracts. There are no typed schemas in the implementation. No Pydantic model, dataclass, `TypedDict` or JSON Schema is defined anywhere; `pydantic` is not imported. The contracts that do exist are structural constants and the shape of the returned dictionaries:

- `olist.EXPECTED_COLUMNS` — the expected column list for each of the nine Olist tables, used as an executable contract by `validate_schema()`.
- `olist.DATE_COLUMNS` — which columns are parsed as timestamps and which are therefore subject to the non-regression gate.
- `online_retail.EXPECTED_COLS` — the exact eight-column contract enforced by `_clean()`.

- `config.OLIST_FILES`, `config.RETAIL_SHEETS`, `config.OLIST_TABLE_ORDER` — the input inventory and presentation order.
- The `frames`/`facts` dictionaries — the informal internal interface between `run()` functions, `__main__` and `figures`. Their keys are implicit: `frames` holds named `DataFrame`s (for example `schema_validation`, `column_profile`, `reconciliation_proxy` facts) and `facts` holds nested dictionaries. Because they are untyped, a typo or a renamed key would fail only at run time.
- The output contract is likewise implicit: the column sets of the CSV artifacts exist only as the code that writes them. No schema file or test asserts them.

What does not exist: agent-facing contracts. There is no request schema, no response schema, no task or workflow-state schema, no anomaly-record schema, no governance-decision schema and no audit-event schema. These are **proposed** in the sibling reports — a data-contract section and an audit-event schema in the Agent Report, and an audit-event schema plus a canonical-event discussion in the Dataset Report — and are not implemented. They are named here only to prevent any implication that they exist in code.

11 Agent Communication and Workflow Orchestration

Mechanism actually used. The only communication mechanism is a direct synchronous Python function call, with results passed as plain dictionaries of `DataFrame`s in the calling process's memory. Read from `main()`, the control flow is:

```
config.ensure_dirs()
olist_tables, olist_raw_dups = olist.load_tables()
ol            = olist.run(olist_tables, olist_raw_dups)
retail_sheets = online_retail.load_sheets()
rt            = online_retail.run(retail_sheets)
dims, ol_dim, rt_dim = _dimensions(olist_tables, retail_sheets)
# 26 x config.write_csv(...) ; figures.*(...) ; config.write_json(...)
```

A small number of minor side-channels exist and are worth recording: `olist.run()` stores private frames under keys prefixed with an underscore (`results["_items"]`, `results["_orders"]`, `results["_cleaned"]`, `results["_combined"]`) so that `figures` can reuse already-parsed data, and `online_retail.run()` similarly exposes `_cleaned` and `_combined`. This is shared in-memory state, not messaging.

Mechanisms not used. No message broker or queue, no HTTP endpoint, no RPC, no graph-based workflow engine, no LLM tool calling, no event bus and no persistence layer. The `PATH`-B inspector is an entirely independent program: it shares no code with the analysis package and reads the raw files again.

Retries, error propagation and conflicting results. There is no retry logic, no timeout, no compensation and no dependency resolution. Error propagation is Python's default: the two validation gates raise `ValueError` with a message naming the file and column, and the process aborts. Because there is exactly one result per metric, there is no concept of conflicting results to reconcile. Any inter-agent messaging, retry or conflict-resolution design lives in the Agent Report and is explicitly proposed.

12 Governance, Security and Audit Implementation

Figure 5 separates the controls that the code actually enforces from those that exist only as documentation. Table 5 then attributes each control to a file or records its absence. The external reference points for the controls a governed AI system is normally expected to provide are the NIST AI Risk Management Framework [22, 1] and ISO/IEC 42001 [11]; this section measures the repository against that expectation.

Figure 5 - Governance and audit controls: enforced by code vs. documented only

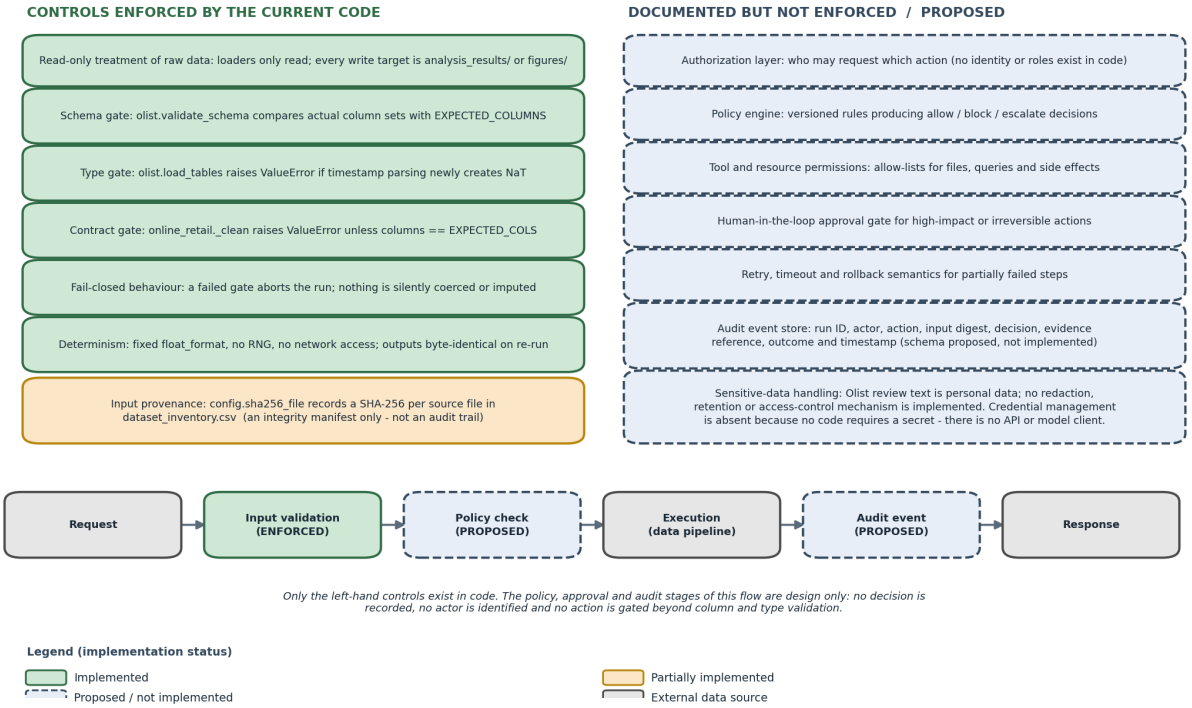


Figure 5: Governance and audit controls. The left column is enforced by code; the right column, and the middle of the flow strip, are design only.

Control	Classification	Implementing file, or reason for absence
Raw-data immutability (by usage)	Enforced by software	`olist.load_tables`, `online_retail.load_sheets`, `inspect_datasets.xlsx_rows` only read; every writer targets `analysis_results/` or `figures/`
Input schema validation	Enforced by software	`olist.validate_schema`; `online_retail._clean`
Timestamp non-regression gate	Enforced by software	`olist.load_tables` raises `ValueError`
Fail-closed abort on invalid structure	Enforced by software	the two `raise` statements; no silent coercion of key columns
Input provenance (integrity manifest)	Partially implemented	`config.sha256_file` via `compare.dataset_inventory`; digest only, no event, no timestamp, no actor
Deterministic, network-free execution	Enforced by software	`config.write_csv` fixed `float_format`; no network or random calls; verified byte-identical re-run
Authorization / access control	Not enforced	no identity, role or permission concept exists in any file
Policy evaluation (allow/block/escalate)	Documented but not enforced	described in the Agent Report; no policy data, rule file or engine in the repository
Tool / resource permissions	Documented but not enforced	no tool layer exists
Human-in-the-loop approval	Documented but not enforced	described in the Agent Report; no approval object or gate in code
Retry / rollback / timeout	Proposed	no retry, timeout or transaction semantics anywhere
Audit-event generation and store	Documented but not enforced	no event log, run ID, actor or decision record; `logging` is never imported
Sensitive-data handling	Documented but not enforced	Olist review text is personal data under regimes such as the GDPR [6]; no redaction, retention or access control is implemented
Credential management	Not applicable / absent	no API client, model provider or database connection exists, so no secret is required or stored

Table 5: Every governance-relevant control found, classified as (1) enforced by the software, (2) documented but not enforced, or (3) proposed/absent. Applying the standard of Section 12 of the task, only the first five rows are technically enforced.

13 Testing and Code Quality

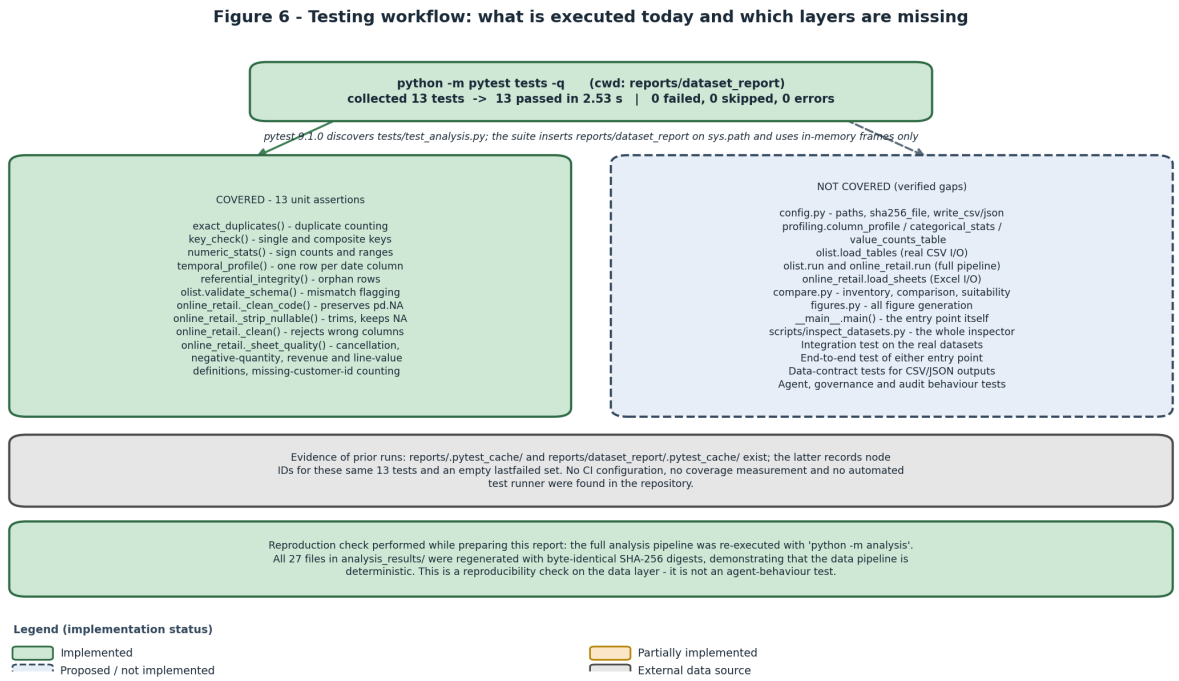


Figure 6: Test coverage as measured. The left column is covered by the existing thirteen assertions; the right column is verified to be uncovered.

Framework and layout. pytest 9.1.0. A single test module, `reports/dataset_report/tests/test_analysis.py` (177 lines, fourteen functions of which thirteen are tests), with no `conftest.py`, no `pytest.ini` and no `tox.ini`. The module inserts `Path(__file__).resolve().parents[1]` on `sys.path` so that `analysis` is importable, which is a workaround for the absence of an installed package.

Command executed and observed result. The following command was run for this report, from `reports/dataset_report`:

```
python -m pytest tests -q
.........                                                    [100%]
13 passed in 1.49s
```

All thirteen tests passed; there were no failures, errors, skips or xfails. The tests use only small in-memory frames and never read the raw datasets, so they are fast and safe.

What the tests actually assert. Missing-value preservation (`_clean_code`, `_strip_nullable`); exact-duplicate counting; composite-key uniqueness and duplicate detection; referential-integrity orphan counting in both directions; sign counts and ranges in `numeric_stats`; one-row-per-column behaviour of `temporal_profile`; cancellation detection and negative-quantity definitions; revenue and line-value definitions; missing-customer-identifier counting; Olist schema-mismatch flagging; and rejection of wrong columns by `_clean`.

Reproducibility check performed. The full pipeline was re-executed with `python -m analysis`. All 27 files in `analysis_results/` were regenerated and every one was byte-identical (SHA-256) to the pre-existing artifact. This is a strong, verifiable statement about the data layer: the pipeline is deterministic. It says nothing about agent behaviour, because there is none.

Test layers that are absent. There are no integration tests (nothing exercises `run()`, `load_tables()`, `load_sheets()` or the real files end to end), no end-to-end tests of either entry point, no data-contract tests for the CSV/JSON outputs, no test of `config`, `compare` or `figures`, and no tests for agents, governance or audit. Figure 6 lists these verified gaps.

Coverage gaps and recommended tests. The most valuable additions, in order, are: (1) an integration test that runs each path on a small fixture directory and asserts output file names, column sets and key invariants; (2) contract tests asserting the column schema of every CSV artifact; (3) tests for `olist.load_tables`'s failure behaviour on a deliberately corrupt timestamp column; (4) a regression test capturing the review-table row count (99,224) so that the inspector defect cannot recur silently; (5) tests for `compare` and `figures` smoke-import; and (6) once agents exist, unit tests for policy decisions and audit-event emission.

Tooling and code-quality observations. There is no linter, formatter, type checker, coverage measurement or CI configuration. Evidence of prior pytest runs exists in `reports/.pytest_cache/` and `reports/dataset_report/.pytest_cache/`; the latter records node IDs for the same thirteen tests and an empty `lastfailed` set, consistent with an all-passing suite. One vestigial assertion exists in the schema test (`assert good is not None`) and one dead statement in the inspector (line 99).

14 Execution and Reproducibility Guide

Every command below was executed during preparation of this report unless explicitly marked otherwise.

1. Prerequisites. Python 3.10 (the verified interpreter is 3.10.11). A LaTeX distribution (MiKTeX 24.1 was verified) is needed only to rebuild the reports.

2. Environment setup. No virtual environment is configured in the repository. Create one if desired; no environment variables are read by any script (verified: `os.environ` is not used except for an implicit default in `inspect_datasets.py`'s argument defaults).

3. Dependency installation. No dependency manifest exists, so installation is inferred from imports and observed versions and is therefore partly *unverified*. The versions below are those observed in the working environment:

```
python -m pip install pandas==2.3.3 numpy==1.26.4 matplotlib==3.10.7 \
    seaborn==0.13.2 openpyxl==3.1.5 pytest==9.1.0
```

The inspector path requires *no* third-party package.

4. Configuration setup. None required. Paths are derived from the file locations; there is no configuration file, no secret and no environment variable to set.

5. Dataset placement. The raw data must remain at `datasets/olist/*.csv` (nine files) and `datasets/online_retail_ii/online_retail_II.xlsx`. The analysis package computes `REPO_ROOT` as the parent of `reports/`, so moving the datasets breaks the Olist loader and the workbook loader. The inspector accepts `-root` and `-output` and is therefore relocatable.

6. Execution commands. Verified:

```
# analysis package (writes analysis_results/ and figures/)
python -m analysis            # run from reports/dataset_report

# independent inspector (stdlib only), run from the repository root
python reports/dataset_selection_table/scripts/inspect_datasets.py

# regenerate the diagrams of this report
python figures/make_figures.py   # from reports/code_report
```

The workbook read is the slow step; the run prints a warning-free progress line and takes roughly one minute on the inspected machine.

7. Test commands. Verified: `python -m pytest tests -q` from `reports/dataset_report` (`13 passed`).

8. Output locations. `reports/dataset_report/analysis_results/` (26 CSV + `summary.json`), `reports/dataset_report/figures/` (13 PNG), `reports/dataset_selection_table/` (five CSV + `inspection_summary.json`), and `reports/code_report/figures/` (six PNG).

9. Troubleshooting. `ModuleNotFoundError: analysis` means the command was run outside `reports/dataset_report`; use the quoted working directory. `ValueError: ...: N values failed timestamp parsing` means an input file was modified or corrupted, and is the intended fail-closed behaviour. `ValueError: sheet '...' columns ... != expected ...` means the workbook structure changed. `ImportError: Missing optional dependency 'openpyxl'` means openpyxl is not installed. If a PDF is to be rebuilt, run the four-pass `pdflatex/bibtex/pdflatex/pdflatex` sequence; a path containing a space such as `reports/Agents reports` must be quoted on Windows.

15 Outputs and Evaluation

15.1 Artifacts produced by the code

Table 6 lists the artifacts. No output is a performance measurement.

Artifact	Format	Generating module	Meaning and intended use
analysis_results/table_dimensions.csv	CSV	__main__._dimensions	Rows and columns per table/sheet
.../olist_schema_validation.csv	CSV	olist.validate_schema	Schema conformance per Olist table
.../olist_column_profile.csv	CSV	profiling.column_profile	Completeness and cardinality per column
.../olist_duplicate_rows.csv	CSV	profiling.exact_duplicates	Raw and parsed duplicate counts
.../olist_numeric_stats.csv	CSV	profiling.numeric_stats	Distribution, quartiles, IQR outlier counts
.../olist_categorical_stats.csv	CSV	profiling.categorical_stats	Top categories and cardinality
.../olist_temporal_profile.csv	CSV	profiling.temporal_profile	Per-column timestamp ranges and spans
.../olist_primary_key_checks.csv	CSV	profiling.key_check	Candidate keys and duplicate excess
.../olist_referential_integrity.csv	CSV	olist.referential_integrity	Orphan rows per foreign key, both directions
.../olist_join_cardinality.csv	CSV	olist.run	Relationship cardinality and child counts
.../olist_join_expansion.csv	CSV	olist._join_expansion	Inner-join row inflation factors
.../olist_value_counts.csv	CSV	profiling.value_counts_table	Status, payment type, review score, state, category
.../olist_category_revenue.csv	CSV	olist.run	Top categories by item value
.../olist_monthly_orders.csv	CSV	olist.run	Order volume by purchase month
.../retail_*.csv (nine files)	CSV	online_retail.run	Workbook profiles, quality metrics, business signals
.../dataset_inventory.csv	CSV	compare.dataset_inventory	File sizes and SHA-256 digests
.../dataset_comparison.csv	CSV	compare.dataset_comparison	Curated comparison (mixed measured and qualitative)
.../agent_suitability.csv	CSV	compare.agent_suitability	Curated per-agent suitability (qualitative)
.../summary.json	JSON	__main__.main	Aggregated facts for both datasets
figures/fig_*.png (13 files)	PNG	figures	Report figures computed from the real data
dataset_selection_table/*.csv	CSV	inspect_datasets.main	Independent inventory, quality, key and suitability tables
dataset_selection_table/inspection_summary.json	JSON	inspect_datasets.main	Machine-readable inventory and relationship checks

Table 6: Artifacts generated by the code. analysis_results contains 26 CSV files plus summary.json; the file list above groups the nine retail CSVs.

15.2 Evaluation status and proposed plan

No evaluation of the multi-agent system has been performed, because no multi-agent system exists. There are no experiment logs, no baselines, no metrics and no results tables for agent behaviour in the repository. The Dataset Report and the Agent Report contain *proposed* evaluation protocols; those remain proposals. This Code Report therefore proposes a code-oriented evaluation plan consistent with what will actually be built:

- **Ingestion and validation layer.** Metrics: schema-gate pass rate on a frozen input corpus; determinism rate (currently 100% over 27 artifacts); median runtime; memory peak. Baselines: the current pipeline.
- **Orchestration.** Metrics: workflow completion rate, dependency-violation count, retry count, end-to-end latency on replayed orders. Baseline: the current manual sequence.
- **Governance.** Metrics: policy-decision accuracy against a hand-labelled rule set, false-block rate, and the fraction of high-impact actions routed to approval. Baseline: column/type validation only.
- **Audit.** Metric: evidence-traceability rate — the fraction of decisions for which every source record referenced can be resolved. Baseline: none (no event log exists).
- **Reproducibility and limitations.** Every reported number must be regenerable by one documented command on a pinned environment; the absence of fraud labels means no detector accuracy will be reported.

16 Limitations and Technical Debt

Findings are separated into *verified defects* (reproduced during this inspection) and *risks*. Severity is an engineering judgement.

ID	Finding	Type / severity	Evidence and recommended fix
D1	`src/` is empty and the root `README.md` is 0 bytes	Verified / high	Directory and file listing; `(Get-Item README.md).Length = 0`. Fix: remove the placeholder or populate it as the real package root and installation entry point.
D2	Reviewer/inspector over-counts the Olist review table by 26 rows	Verified defect / high	`csv.Sniffer` yields `doublequote=False`, producing 99,250 rows versus the correct 99,224 (pandas and default `csv.DictReader` agree). The error propagates to the inspector's `dataset_inventory.csv` and `inspection_summary.json`, and `main.tex` displays the inflated 99,250. Fix: pass an explicit dialect with `doublequote=True`, or drop `Sniffer` in favour of a fixed dialect.
D3	No dependency manifest	Verified / high	No `requirements.txt`, `pyproject.toml` or lock file. Fix: add a pinned `requirements.txt` (or `pyproject.toml`) with the versions in Table 1.
D4	No output validation or assertion on generated artifacts	Verified / medium	`main()` writes and prints; nothing checks schema or counts. Fix: add a post-write contract check.
D5	No logging; progress via `print()`	Verified / medium	`logging` is never imported. Fix: adopt `logging` with a run identifier.
D6	Analysis package has no CLI	Verified / low	No `argparse` in `analysis`; inputs are hard-coded lists. Fix: add `-olist-dir`, `-retail-xlsx`, `-out`, `-figures`.
D7	Bare `except` clauses and dead code in the inspector	Verified / low–medium	Lines 50–54, 94–95, 107–129 swallow errors; line 99 builds an unused `Counter`. Fix: narrow exception types and delete the dead statement.
D8	Fragile path derivation in the inspector	Verified / low	`ROOT = Path(__file__).resolve().parents[3]`. Fix: use `-root` or resolve upward until a marker file is found.
D9	Curated text mixed with measured data in one artifact	Verified / medium	`compare.dataset_comparison` and `agent_suitability` contain hand-written strings; the docstring acknowledges this. Fix: separate measured and interpretive columns, or mark them explicitly.
D10	Whole-dataset memory load	Verified / low	`read_csv(low_memory=False)` and `read_excel(sheet_name=None)` load both sheets and all tables at once. Fix: chunking or per-sheet streaming if the corpus grows.
D11	Code and generated artifacts share one directory	Verified / low	`analysis/` sits beside `analysis_results/` and `figures/`. Fix: move the package to `src/` and treat `reports/` as output only.
D12	Test-module `sys.path` hack and vestigial assertion	Verified / low	`sys.path.insert` at test import; `assert good is not None`. Fix: install the package or add `conftest.py`; delete the no-op assertion.
R1	No governance/audit primitives at all	Risk / high	Required before any agent can act under policy. Fix: implement the policy gate and the audit store first (roadmap phase 3).
R2	No fraud labels, no stock data, no ledger	Risk / medium	Stated in both datasets and in `agent_suitability`. Fix: keep claims scoped; treat outputs as anomaly candidates.
R3	Two divergent readers of the same corpus	Risk / medium	`analysis` (pandas) and `inspect_datasets` (stdlib) duplicate profiling logic and have already diverged (D2). Fix: keep one canonical implementation and derive the other as a thin adapter.
R4	No CI or automated runner	Risk / medium	No workflow file. Fix: add a minimal CI job running the tests and a determinism check.

Table 7: Prioritized findings. D2, D3, D4 and R1 have the highest immediate impact on correctness and on the roadmap.

Note on integrity claims. Because the code has no event log and no integrity primitive beyond the file digests, the project must not describe the current system as auditable, tamper-evident or compliant. Those properties are proposed, not present.

17 Development Roadmap

The phases are ordered so that nothing already implemented is rebuilt, and so that governance and audit exist before any agent is allowed to act. Effort estimates are engineering judgements, not commitments.

Phase	Goal	Deliverable	Depends on
1	Stabilise the data pipeline	Fix D2; add D4 output contracts; fix D7/D8 in the inspector; freeze a small fixture corpus	nothing
2	Pin the environment and package the code	Add `requirements.txt` (D3); move the package to `src/` (D11, D1) with `pyproject.toml`; remove the test `sys.path` hack	phase 1
3	Implement the governance gate and the audit store first	Versioned policy rule set, allow/block/escalate decision, append-only event writer with run IDs and evidence references	phase 2
4	Define and freeze the typed data contracts	Pydantic or dataclass models for canonical events, agent requests/responses, decisions and audit events, each with tests	phase 2
5	Complete the orchestrator	Task object, deterministic planner, specialist dispatch, dependency resolution, failure propagation, result consolidation — all routed through phase 3 gates	phases 3–4
6	Implement the specialist agents one at a time	Reuse the existing data layer directly: E-Commerce on `olist.run` outputs; Accounting on the reconciliation proxy; Risk on the IQR/negative-quantity signals; Inventory strictly as a demand-indicator agent with an explicit data-gap declaration	phase 5
7	Add integration and end-to-end tests	Fixture-based pipelines, contract tests, scenario tests for approval, rejection and audit traceability	phases 5–6
8	Establish reproducible evaluation	The metrics in Section 15, with baselines and a one-command harness	phase 7
9	Documentation and deployment readiness	Replace the empty README; document the contracts; add CI (R4); containerise if required	phases 2–8

Table 8: Prioritized roadmap. Phases 3 and 4 deliberately precede agent implementation: without a policy gate and an audit store, no agent action would be governable or traceable.

18 Conclusion

What has been implemented. A reproducible dataset analysis, validation and reporting pipeline of 2,144 lines of Python across eleven files, comprising the package `reports/dataset_report/analysis/` with seven modules and a `python -m analysis` entry point, an independent standard-library inspector, and two figure generators for the sibling reports. The pipeline reads both research datasets read-only, applies two fail-closed structural gates, profiles nine Olist tables and two workbook sheets, measures keys, referential integrity, joins, distributions, temporal and monetary properties, derives a payment-versus-item reconciliation proxy and several workbook business signals, and exports 26 CSV files, one JSON summary and thirteen figures.

What currently works. The pipeline and the inspector execute successfully; the thirteen unit tests pass (`13 passed in 1.49s`); the pipeline is deterministic, with all 27 regenerated artifacts byte-identical; and the code correctly refuses to invent stock levels, ledger entries, policies or fraud labels. The repository also contains three coherent LaTeX reports.

What has been tested. Only the data layer, and only at helper level: thirteen assertions over missing-value handling, duplicate counting, key checks, referential integrity, sign counts, temporal profiling, cancellation and revenue definitions, and schema validation. There is no integration, end-to-end, contract, agent, governance or audit test.

What remains incomplete. Everything above the ingestion and analysis layer. The repository has zero classes, no orchestrator, no messaging, no typed contracts, no policy engine, no approval mechanism and no audit event store; `src/` is empty and the root README is empty. All seven agents are *designed but not implemented.* Three defects were verified during this inspection, the most concrete being the inspector's 26-row over-count of the Olist review table caused by an incorrectly inferred CSV dialect.

How the code supports the research objectives. The pipeline is precisely the ingestion and validation layer that the proposed architecture requires: it establishes stable, tested, reproducible access to both datasets and produces trustworthy measured facts that the sibling reports reuse. It gives the project a credible foundation and a clear, verified statement of the distance to a functioning governed multi-agent system. The roadmap in Section 17 closes that distance in a disciplined order, with governance and audit implemented before any agent is permitted to act.

19 References and Technical Documentation

The bibliography below reuses the references already present in the sibling reports' `references.bib` files and adds official documentation for the libraries that the code actually imports. Every entry was checked at the time of writing: the eight software-documentation URLs, the two dataset landing pages and the dataset DOIs returned HTTP 200, and the Crossref metadata for the journal, conference and NIST entries was retrieved and matched by DOI. The ISO catalogue page for ISO/IEC 42001 is the canonical address but refuses automated requests, as do some publisher pages; these respond normally in a browser. No citation, author, year, DOI or URL has been invented.

References

[1] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024.

[2] Carlo Batini, Cinzia Cappiello, Chiara Francalanci, and Andrea Maurino. Methodologies for Data Quality Assessment and Improvement. *ACM Computing Surveys*, 41(3):1–52, 2009.

[3] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

[4] Daqing Chen. Online Retail II, 2012. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0); 1,067,371 instances.

[5] Committee of Sponsoring Organizations of the Treadway Commission (COSO). Internal Control — Integrated Framework. Framework, 2013.

[6] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data (General Data Protection Regulation). Official Journal of the European Union, L 119, 2016.

[7] Eric Gazoni and Charlie Clark. openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files. Online documentation, 2024. Used as the `engine="openpyxl"` backend for `pandas.read_excel` in `online_retail.load_sheets()`.

[8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[9] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design Science in Information Systems Research. *MIS Quarterly*, 28(1):75–105, 2004.

[10] John D. Hunter. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[11] International Organization for Standardization and International Electrotechnical Commission. ISO/IEC 42001:2023 Information Technology — Artificial Intelligence — Management System. International Standard, Edition 1, 2023. Published 2023-12; prepared by ISO/IEC JTC 1/SC 42.

[12] Matplotlib Development Team. Matplotlib Documentation. Online documentation, 2024.

[13] Wes McKinney. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. Reference publication for the pandas library.

[14] MiKTeX Project. MiKTeX: a free cross-platform LaTeX distribution. Online documentation, 2024. MiKTeX-pdfTeX 4.18 (MiKTeX 24.1) with BibTeX 4.1 was used to compile this report and its sibling reports.

[15] Eric W. T. Ngai, Yong Hu, Y. H. Wong, Yijun Chen, and Xin Sun. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569, 2011.

[16] NumPy Developers. NumPy Reference — Documentation. Online documentation, 2024.

[17] Olist. Brazilian E-Commerce Public Dataset by Olist. Online dataset, 2018. Nine relational CSV files; licence as stated on the Kaggle data card; accessed via local copy.

[18] pytest development team. pytest: helps you write better programs — Documentation. Online documentation, 2024. Version 9.1.0 was used to execute the 13 unit tests of the analysis package.

[19] Python Software Foundation. The Python Language Reference, version 3.10. Online documentation, 2024. CPython 3.10.11 was the interpreter observed in the analysis environment.

[20] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, USA, 4 edition, 2021.

[21] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, New York, USA, 2009.

[22] Elham Tabassi. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.

[23] The pandas development team. pandas: powerful Python data analysis toolkit — Documentation. Online documentation, 2024. Version 2.3.3 was installed in the inspected environment; `read_csv`, `read_excel`, `to_datetime` and `to_numeric` are used by the analysis package.

[24] United States Congress. Sarbanes–Oxley Act of 2002. Public Law 107–204, 116 Stat. 745, 2002.

[25] Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021.

[26] Michael L. Waskom. seaborn: statistical data visualization — Documentation. Online documentation, 2024.

[27] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Scientific Data*, 3:160018, 2016.

[28] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, Chichester, UK, 2 edition, 2009.

A Repository Inventory and Verification Log

A.1 Code inventory

Eleven Python files, 2,144 lines, 65 functions, 0 classes (Table 3). Seven analysis modules plus `__init__` and `__main__`; one test module; one inspector; one figure generator for the Agent Report. No dependency manifest, no CI configuration, no container definition, no environment-variable usage, no credential material.

A.2 Commands executed for this report

- `python -version` → `Python 3.10.11`; `pip list` → the versions in Table 1.
- `pdflatex -version` → MiKTeX-pdfTeX 4.18 (MiKTeX 24.1); `bibtex -version` → MiKTeX-BibTeX 4.1.
- `cd reports/dataset_report; python -m pytest tests -q` → `13 passed in 1.49s`.
- `cd reports/dataset_report; python -m analysis` → completed; 26 CSV + `summary.json` + 13 PNG written; all 27 `analysis_results` files byte-identical to the pre-existing artifacts (SHA-256).
- `python reports/dataset_selection_table/scripts/inspect_datasets.py -output <temp>` → inspected 10 data files (nine Olist CSVs, one workbook) and wrote its artifacts to a temporary directory, leaving the committed artifacts untouched.
- `python reports/code_report/figures/make_figures.py` → six PNGs.

A.3 Defect reproduction (D2)

```
csv.Sniffer().sniff(sample).doublequote      -> False
sum(1 for _ in csv.DictReader(open(reviews), dialect=sniffed))  -> 99250
sum(1 for _ in csv.DictReader(open(reviews)))                   -> 99224
len(pandas.read_csv(reviews, low_memory=False))                 -> 99224
```

The 26-row discrepancy is therefore produced by dialect detection, not by the file.

A.4 Integrity of sibling reports

`main.tex`/`main.pdf`, `dataset_report.tex`/`dataset_report.pdf` and `Agent_Report.tex`/`Agent_Report.pdf` were read but not modified while preparing this report. The only written locations are `reports/code_report/` (this report and its figures) and the regenerated deterministic artifacts under `reports/dataset_report/` and the temporary inspector output. No source file was altered.